

PROGRAMMING FOR PROBLEM SOLVING (BE01000121)



Prepared By,

Prof. Shraddha Modi

Computer Engineering

LDCE, Ahmedabad

CHAPTER-7

POINTERS

Prepared By,
Prof. Shraddha Modi
Computer Engineering
LDCE, Ahmedabad

ROADMAP

- ❑ Basics of pointers
- ❑ Pointer to pointer, pointer and array
- ❑ Pointer to array, array to pointer
- ❑ Function returning pointer

INTRODUCTION

- ❖ A normal variable is used to store value.
- ❖ A pointer is a variable that store address / reference of another variable.
- ❖ Pointer is derived data type in C language.
- ❖ A pointer contains the memory address of that variable as their value. Pointers are also called address variables because they contain the addresses of other variables.

INTRODUCTION

- **&** is address of or referencing operator which returns memory address of variable.
- ***** is indirection or dereferencing operator which returns value stored at that memory address.
- **&** operator is the inverse of ***** operator
- **$x = a$ is same as $x = *(&a)$**

INTRODUCTION

Normal Variable

use to
store data

Pointer Variable

use to
store address

Normal Variable

john

100

10010

Pointer Variable

ptr

10010

10060

INTRODUCTION

STEP - 1

simple variable

Declaration of Pointers

pointer variable

data_type var_name

char a;

int b;

float c;

data_type *var_name

char *a;

int *b;

float *c;

STEP - 2

Initialization

&

int john = 10;

int *ptr;

ptr = &john

ptr

11000

10060

john

10

11000

2

INTRODUCTION

STEP-3

Dereferencing

```
printf("%d",ptr);
```

11000

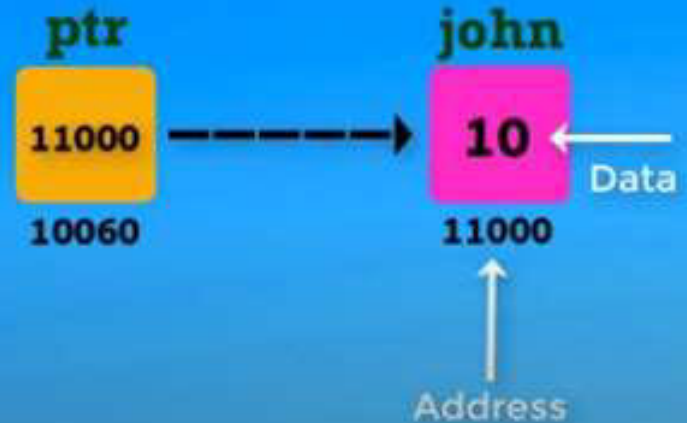
```
printf("%d",&ptr);
```

10060

*

← Dereferencing operator

```
printf("%d",*ptr);
```



INTRODUCTION

```
#include <stdio.h>
void main()
{
    int a = 10;
    int *ptr = &a;
    printf("value of a = %d\n", a);
    printf("value stored at ptr = %d\n", *ptr);
    printf("Address of a = %d\n", &a);
    printf("ptr points to the address = %d\n", ptr);
    printf("Address of ptr = %d\n", &ptr);
}
```

```
value of a = 10
value stored at ptr = 10
Address of a = 1145633052
ptr points to the address = 1145633052
Address of ptr = 1145633040
```

INTRODUCTION

```
#include <stdio.h>
```

```
void main() {
```

```
int x = 10, y,*p1;
```

```
p1 = &x;
```

```
y = *p1;
```

```
printf ("value of x %d\n", x);
```

```
printf ("%d is stored at address %u\n", x, &x );
```

```
printf ("%d is stored at address %u\n", *&x, &x );
```

```
printf ("%d is stored at address %u\n", *p1, p1);
```

```
printf ("%d is stored at address %u\n", p1, &p1 );
```

```
printf ("%d is stored at address %u\n", y, &y );
```

```
*p1 = 100;
```

```
printf ("Now X = %d", x);    }
```

```
value of x 10
```

```
10 is stored at address 1763567980
```

```
10 is stored at address 1763567980
```

```
10 is stored at address 1763567980
```

```
1763567980 is stored at address 1763567968
```

```
10 is stored at address 1763567976
```

```
Now X = 100
```


POINTER TO POINTER – DOUBLE POINTER

- Pointer holds the address of another variable of same type.
- When a pointer holds the address of another pointer then such type of pointer is known as pointer-to-pointer or double pointer.
- The first pointer contains the address of the second pointer, which points to the location that contains the actual value.

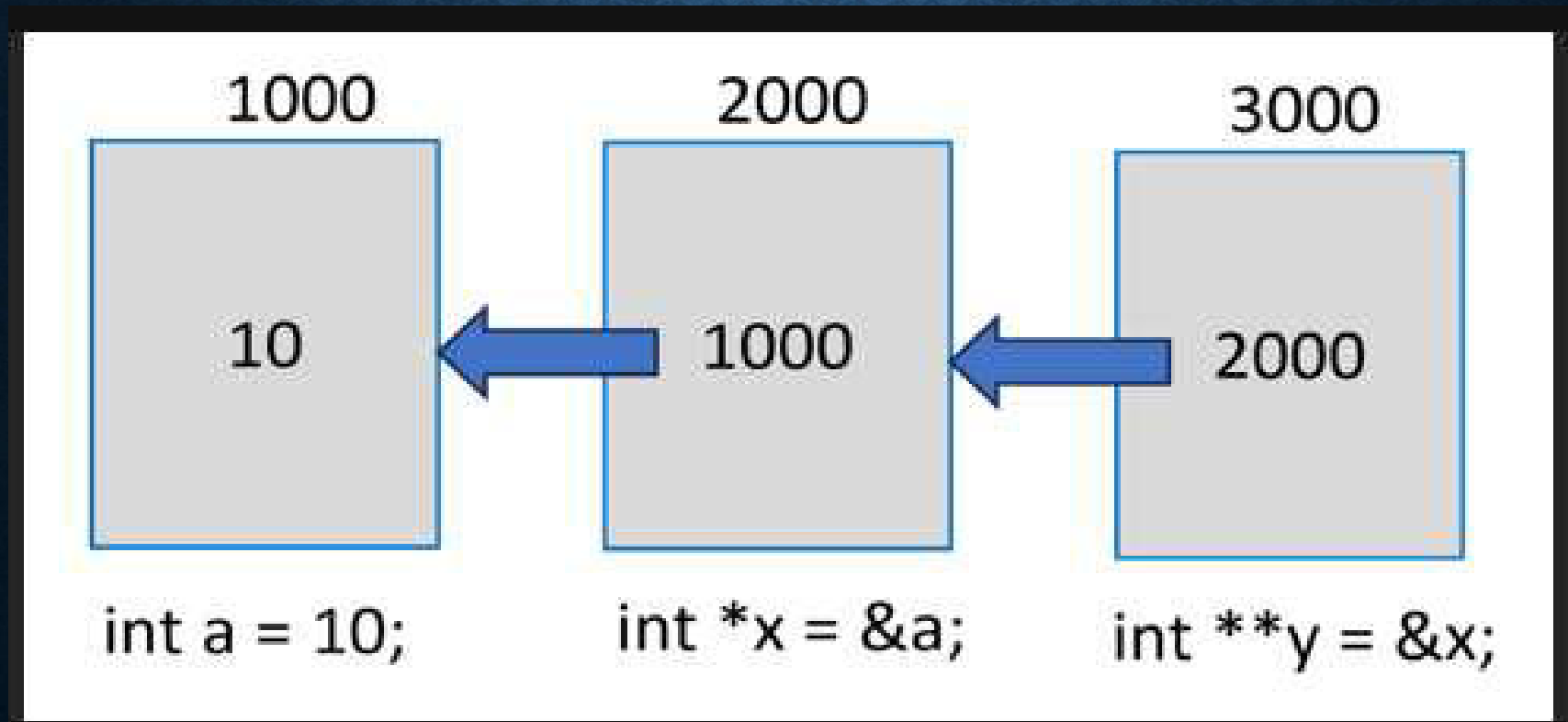
POINTER TO POINTER – DOUBLE POINTER

```
datatype **ptr_variablename;
```

```
int **ptr;
```



POINTER TO POINTER – DOUBLE POINTER



POINTERS AND ARRAYS

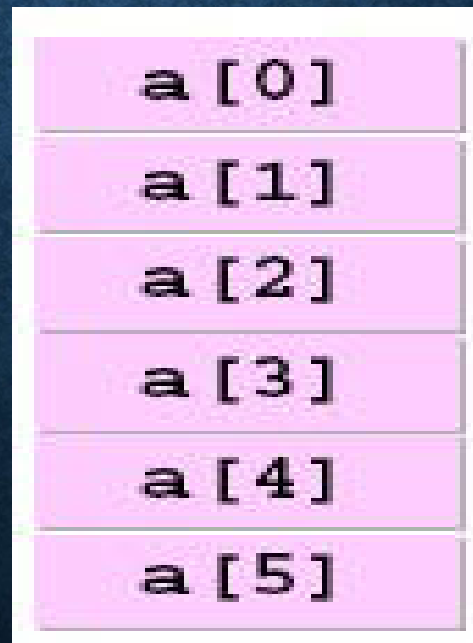
- The following two statements **do the exact same thing**:

`a[5]=56;`

`*(a+5)=56;`

Here is the layout in memory:

- `a` 133268
- `a+1` 133272
- `a+2` 133276
- `a+3` 133280
- `a+4` 133284
- `a+5` 133288



POINTERS AND ARRAYS

- As we have an array of char, int, float etc, same way we can have an array of pointer.
- Individual elements of an array will store the address values.
- So, an array is a collection of values of similar type. It can also be a collection of references of similar type known by single name.

```
datatype *name[size];
```

```
int *ptr[5]; //declares an array of integer  
pointer of size 5
```

POINTERS AND ARRAYS

```
void main()
{
    int *p , sum=0 , i = 0;

    int x[5] = {5, 9, 6, 3, 7} ;

    p = x;

    while(i < 5)
    {
        sum = sum + *p;

        i++, p++;

        printf("sum = %d", sum);

        printf("&x[0] = %u", &x[0]);

        printf("p = %u", p);

    }

    printf(" x[%d] %d %u ", i, *p, p);
}
```


POINTERS AND ARRAYS

Output:

x[0] 5 166

x[1] 9 168

x[2] 6 170

x[3] 3 172

x[4] 7 174

sum = 55

&x[0] = 166

p = 176

POINTERS AND STRINGS

```
void main()  
{  
    char *name;  
    int length;  
    name = "Delhi";  
    char *cptr = name;  
    printf("%s", name);
```

```
    while (*cptr != '\0' )  
    {  
        printf(" %c is stored at address  
                %u ", *cptr, cptr);  
        cptr++ ;  
    }  
    length = cptr - name;  
    printf ("Length of string = %d",  
            length); }
```


POINTERS AND STRINGS

Output:

Delhi

D is stored at address 54

e is stored at address 55

l is stored at address 56

h is stored at address 57

i is stored at address 58

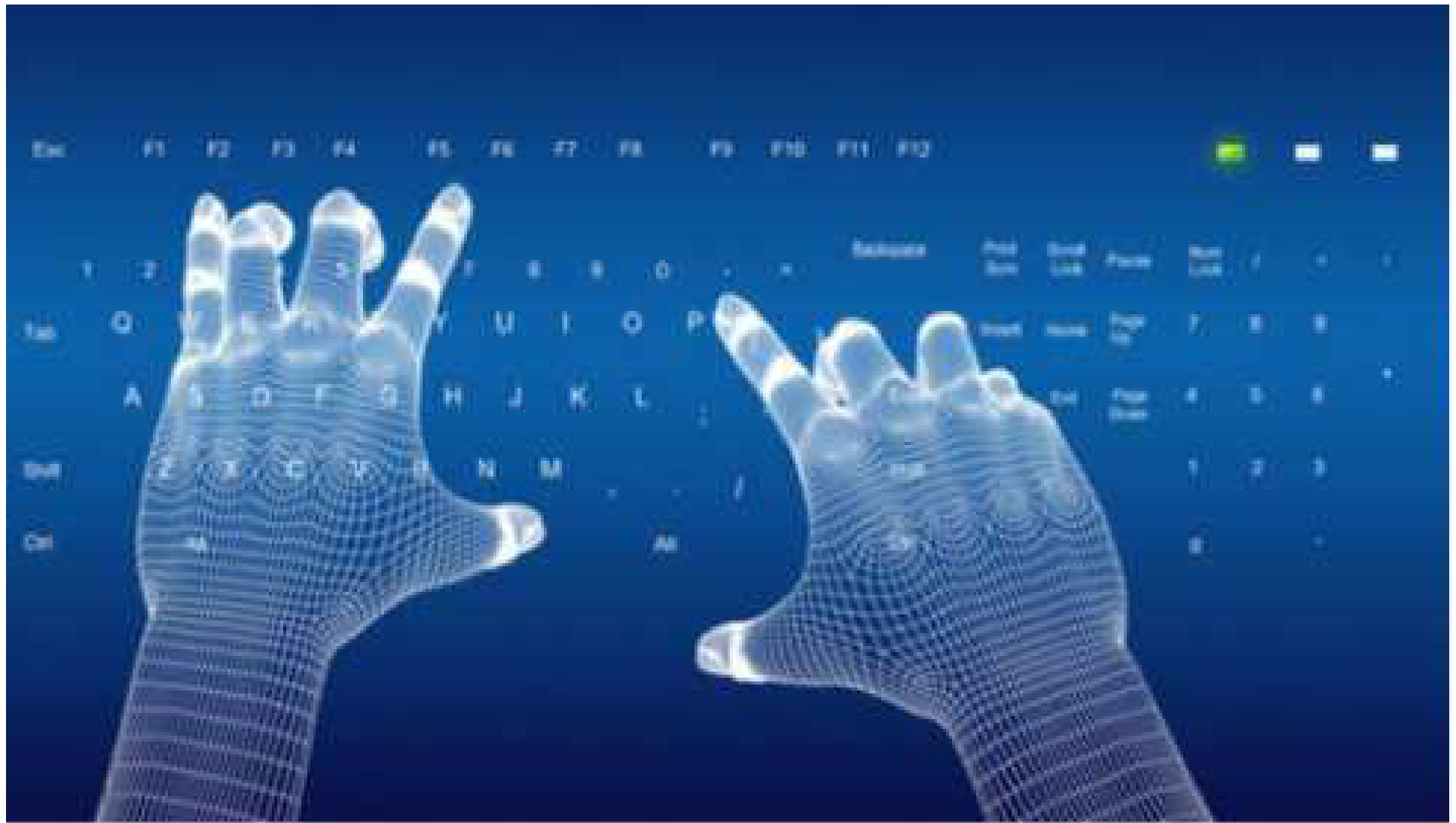
Length of string = 5

POINTER AND FUNCTION

- ❖ Already done in Unit-5 that is call by value and call by function.

FUNCTION RETURNING POINTER

```
int *larger (int *, int *);  
  
void main ()  
{  
  
int a = 10, b = 20;, *p;  
  
p = larger(&a, &b);  
  
printf("%d", *p);  
  
}  
  
int *larger (int *, int *)  
{  
  
if(*x > *y)  
return (x);  
  
else  
return (y);  
  
}
```



Thank You !